

Instructions:

- Solve the following problems to the best of your ability. The problems are challenging, by design—it's ok if you cannot solve them all. If you are stuck, **stop by Office Hours** for help getting unstuck.
- Submit your completed solutions on **Gradescope**. Solutions should be typeset (using LaTeX) as a single pdf file. For each problem, mark the start and end of your solution within the file.
- Collaboration is encouraged while solving the problems! You may discuss the problems with anyone in the class, but significant collaborations (at the level of drawing diagrams or writing out ideas) should be limited to groups of at most 3 students. Additionally, you must:
 1. list the names of all (up to 2) significant collaborators;
 2. write up your solutions, independently from collaborators, in your own words.Do **not** solicit help from people outside of the class (i.e., beyond classmates and course staff).
- If a problem asks you to **design and analyze** an algorithm, you must:
 1. Clearly describe your algorithm (in English and/or pseudocode).
 2. Prove the algorithm's **correctness**.
 3. Analyze the algorithm's asymptotic **running time**.
- In your solutions, you may appeal to any algorithm from this semester's CS 4820 lectures, textbook, and assigned readings, without re-proving correctness and running time. If you use a lemma from 4820 materials or a prerequisite class (CS 1110, 2110, 2800, 3110) without proof, you must **cite your source**.

(1) (8 points) Campsite Assignments.

Treman State Park is unveiling a new set of scenic campsites. Given the popular demand, you've been hired to manage the campsite assignment system. Each weekend, there are n camping groups who want to visit every one of n scenic sites, split across Saturday and Sunday. Every group g submits an itinerary consisting of the following information:

- a starting time s_g on Saturday, when the group g will leave from the parking lot, denoted as c_0 ;
- an ordered list L_g of all the scenic sites, where $L_g[i] = c_j$ indicates that the site c_j will be the i th site visited by group g ;
- an ordered list of times T_g , where $T_g[i]$ indicates the time that group g will take to travel from site $L_g[i-1]$ to $L_g[i]$.¹

Your job is to assign groups to camping sites respecting a few key constraints:

- Exactly one group is assigned to each site.
- Once a group arrives at their assigned campsite, they stop for the night, resuming the itinerary in the morning.
- To respect each group's privacy, once a group has stopped at their assigned campsite c for the night, no other group should pass through site c , until the following day.²

Design and Analyze a polynomial-time **reduction** from the problem of campsite assignments to the STABLEMATCHING problem.

¹You should assume that every itinerary is short enough that, in principle, it could be completed in a single day. Nevertheless, every group wants to camp overnight at one of the campsites.

²It is not a privacy violation, however, if two groups arrive at a campsite at the same time, and one stays to camp while the other continues hiking.

Solution.

Algorithm.

We reduce to the STABLE-MATCHING-PROBLEM by constructing preferences for the groups and sites:

- group g prefers campsites in ascending order according to L_g (i.e. earliest site is most preferred);
- site S prefers a group g to a group h if group g plans to arrive at S *later* than group h .

If there are any ties in the preference list for sites, we will just break them arbitrarily. We then return the assignment of groups to sites obtained by running Gale-Shapley on these preference lists with groups being proposers & sites being receivers (or vice versa).

Correctness.

First, note that Gale-Shapley, given strict preference lists, always returns a stable matching. So, our algorithm always terminates. Next, we wish to show that the assignment of groups to sites is privacy-preserving.

Lemma 1.1. *The stable matching returned by the algorithm respects privacy constraints.*

Proof. Suppose for contradiction that the assignment returned violates the privacy constraint. That is the matching contains the pairs (g, s) and (g', s') however group g , before it reaches s , travels through s' after g' has arrived. In terms of the preferences we construct, this implies that $g \succ_{s'} g'$ and $s' \succ_g s$ which implies that (g, s') is an instability. This is a contradiction! $\square$

Runtime.

Each group's preferences are already specified by the provided itinerary.

We have to do more work to construct the preference list for each site. We start by computing for every group g , its arrival time at a_i^G for each site $L_g[i]$. This can be done by making a linear pass over T_g to computing $a_i^G = s_g + \sum_{j \leq i} T_g[j]$. Then for every group g , add (g, a_i^G) to a list associated with site $L_g[i]$. Finally, we sort the list for each site based on descending arrival times.

The linear pass takes $\mathcal{O}(n)$ and we sort n different lists, which takes $\mathcal{O}(n^2 \log n)$ overall. Running Gale-Shapley itself takes $\mathcal{O}(n^2)$. So, the overall running time is $\mathcal{O}(n^2 \log n)$.

(2) (10 points) Implementing the Reduction.

In this problem, you are asked to code up portions of the Gale-Shapley algorithm for the `STABLEMATCHING` problem, and your reduction from (1). To help with the task, we provide starter code in a file called `main.py`. The starter code provides the implementation of key classes that will be helpful for implementing Gale-Shapley algorithm. There are two comment blocks that include the phrase, `#YOUR CODE HERE#`, where you should add your implementation of:

- The core logic of the Gale-Shapley algorithm; and
- Your reduction from (1).

In all, you will complete a program that solves the campsite assignment problem.

You are responsible for testing your code for correctness! We provide a few public test cases, which are primarily meant to check for correct input-output formatting. Please be aware that these tests are **not** exhaustive. Your code will be subjected to more rigorous private test cases during grading.

Input Format. The input to campsite assignment problem is given in the following format:

- The first line is an integer n , the number of groups/campsites;
- Then, of the next n lines, the g th line specifies the ordered list of campsites L_g for group $g = 1, \dots, n$;
- Then, of the next n lines, the g th line specifies the start time s_g followed by the travel times T_g for group $g = 1, \dots, n$.

The starter code provides a function for reading the input into useful data structures.

Example input. Consider the following instance with 2 groups $\{A, B\}$ and sites $\{c, d\}$. To specify the instance, we give the location lists, the start times, and travel time lists for each group.

$$\begin{array}{llll} \text{Group } A : & L_A = \langle c, d \rangle & s_A = 8 & \text{and } T_A = \langle 1, 2 \rangle \\ \text{Group } B : & L_B = \langle c, d \rangle & s_B = 9 & \text{and } T_B = \langle 2, 3 \rangle \end{array}$$

Given these groups, sites, and lists, the input would be specified as follows.

```
2
1 2
1 2
8 1 2
9 2 3
```

The first line tells us there are $n = 2$ camping groups (A and B) and 2 campsites (c and d). The next two lines describe L_A and L_B (where c and d are encoded numerically as 1 and 2). The final two lines encode s_A followed by T_A and s_B followed by T_B .

Output Format. Given n groups and campsites, the output format consists of n lines, where the g th line begins with the number g followed by the number of the campsite that group g is assigned. The starter code provides a function that handles writing the assignment to `stdout`. Be careful to avoid extra spaces or any other extraneous output!

Submission. Submit your assignment on Gradescope as a single file called `main.py`.

(3) (12 points) Domino Tilings.

In Lecture 1, we saw the definition of a tiling: a *tiling* of a shape S with a tile t is an arrangement of non-overlapping copies of t that exactly cover the area of S . In particular, we considered the DOMINOTILING Problem, where dominoes are tiles of dimension 2×1 (or rotated, 1×2).

We say a shape S is a *sub-grid* if its area can be broken down into contiguous 1×1 squares, which are aligned on an integer grid. In particular, the shape S can be described by a collection of integer coordinates $(x, y) \in \mathbb{Z}^2$, indicating the lower left corner of a unit square contained by S .

The DOMINOTILING Problem asks:

Given a sub-grid shape S , does there exist a tiling of S using dominos?

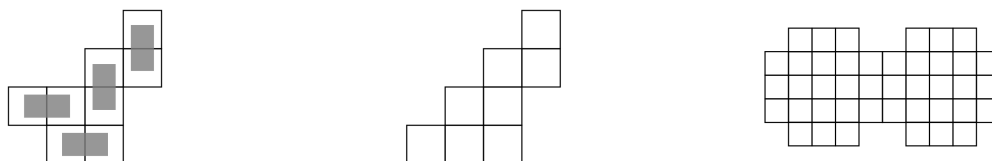


Figure 1: Example DOMINOTILING instances. The shape on the far left CAN be tiled with dominos, whereas the shapes in the center and on the far right CANNOT be tiled with dominos.

Design and Analyze a polynomial-time algorithm to solve the DOMINOTILING Problem.

Assume that the input is a sub-grid shape specified by a set of coordinates.

$$S = \{(x, y) : 1 \times 1 \text{ square area starting at } (x, y) \text{ is included in the shape} \}$$

Concretely, the center example from Figure 1 could be specified as follows.

$$S = \{(0, 0), (1, 0), (2, 0), (1, 1), (2, 1), (2, 2), (3, 2), (3, 3)\}$$

Your algorithm should run in polynomial time in the area of the shape S .

Solution.

Algorithm.

We start by constructing a graph G with vertex set $V = S$ and edgeset

$$E = \{(x, y) \subseteq S^2 \mid |x_1 - y_1| + |x_2 - y_2| = 1\}.$$

Intuitively, G is just a convenient representation of our board as a “grid graph”. Each vertex is a tile and the condition that edges only exist between tiles whose coordinates differ by exactly 1 in one of the directions corresponds to tiles that are adjacent to one another. It is also useful to think of these edges as being *potential* dominoes we may place.

We then partition $V = W \sqcup B$ via the following rule

$$(x_1, x_2) \in W \text{ iff } x_1 + x_2 = 0 \pmod{2}.$$

Finally, we compute $M = \text{MAX-BIPARTITE-MATCHING}(G)$. If $|M| \geq |S|/2$ then our algorithm claims that a tiling exists, otherwise we say that no valid tiling exists.

Correctness.

First, we should verify that G is bipartite so that the reduction to MAX-BIPARTITE-MATCHING even makes sense.

Lemma 3.2. *G is bipartite.*

Proof. Suppose that $(x, y) \in E$ then we have, by construction of our edge set,

$$(x_1 + x_2) - (y_1 + y_2) = \pm 1 = 1 \pmod{2}.$$

So, x and y cannot be in the same set of our partition $V = W \sqcup B$. □

Note now that the condition that $|M| \geq |S|/2$ is equivalent to saying that M is a perfect matching. This follows from noting that $2|M|$ is the number of vertices that are adjacent to an edge in the matching M and $|S|$ is the total number of vertices in G .

We now finish proving correctness of the reduction by showing a one-to-one correspondence between the instances of our two problems.

Lemma 3.3. *If there exists a perfect matching M on the graph G then there is a valid tiling of the S by dominoes.*

Proof. Given the perfect matching M , each edge $e \in M$ identifies a domino— particularly, by identifying the two tiles covered by the domino. Since edges only exist between adjacent tiles, we only identify valid dominoes. Furthermore, note that as M is a perfect matching, every vertex in $V = S$ is adjacent to exactly one edge in E . In terms of our dominoes, this implies that every tile is covered by exactly one domino. That is, the entire shape is tiled with dominoes and no two dominoes overlap. □

Lemma 3.4. *If there exists a valid tiling of S by dominoes then there exists a perfect matching M on the graph G .*

Proof. Let T be such a tiling of dominoes that exactly covers S . Define $M \subseteq E$ as follows:

$$M = \{(x, y) \in E \mid \text{there is a domino in } T \text{ covering the tiles } x, y\}.$$

This is well defined as dominoes can only cover adjacent tiles and edges in G only exist between adjacent tiles. Furthermore, note that as no two dominoes cover the same tile, no two edges in M are adjacent to the same vertex. By definition, this makes M a matching. As every tile $x \in S$ in our shape is covered by some domino in T covering x and some other tile y , we note that every vertex in V is adjacent to an edge $(x, y) \in M$. Thus, M is a perfect matching. □

Runtime.

Let $n = |V| = |S|$.

Constructing the graph G takes $\mathcal{O}(n)$: The vertex set V is just S . Computing the edgeset also takes $\mathcal{O}(n)$ as we may just iterate over every vertex and create (at most) 4 edges for it.

The runtime of MAX-BIPARTITE-MATCHING(G) is $\mathcal{O}(n^2)$.

Thus, the overall runtime of our reduction is $\mathcal{O}(n^2)$.